

Git for Developers

Module 3 — Branching & Merging

Parallel Development, Conflict Resolution, and History Visualization

Module 3 — Agenda

Branching Fundamentals

- What is a branch? How Git implements branches
- Creating and switching branches
- Merging basics: integrating branch work

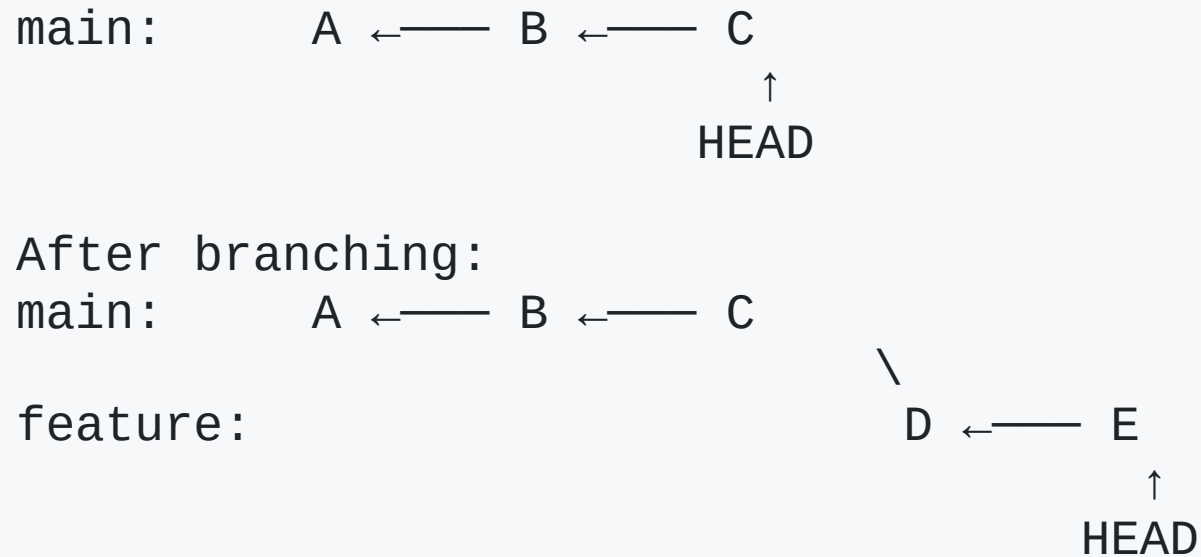
Merging in Depth

- Fast-forward vs. 3-way merge
- Merge conflicts: understanding and resolving them
- Visualizing branches with `git log --graph`

Part 1 — What is a Branch?

What is a Branch?

A branch is a **lightweight, movable pointer** to a specific commit.



Creating a branch does not copy files. It just creates a new pointer — instant and free.

Why Branches?

Use case	How branches help
New feature	Work in isolation without breaking <code>main</code>
Bug fix	Branch from production, fix, merge back
Experimentation	Try ideas — delete the branch if it doesn't work
Code review	Open a pull request from branch → <code>main</code>
Release management	Maintain separate release lines

Good branching strategy = clean history + safe deployments + faster reviews.

How Git Stores Branches

A branch is nothing more than a 41-byte file containing a commit SHA:

```
$ cat .git/refs/heads/main  
a3f7c218b9e4c1d2f3a4b5c6d7e8f9a0b1c2d3e4  
  
$ cat .git/HEAD  
ref: refs/heads/main
```

- **HEAD** points to the current branch (or directly to a commit in "detached HEAD" state)
- Switching branches just updates the **HEAD** file and restores the working directory

Part 2 — Creating & Switching Branches

Creating Branches

```
# List all local branches (* marks the current branch)
```

```
git branch
```

```
# List all branches including remote-tracking branches
```

```
git branch -a
```

```
# Create a new branch (does not switch to it)
```

```
git branch feature/user-auth
```

```
# Create and switch in one command (preferred)
```

```
git checkout -b feature/user-auth
```

```
# Modern equivalent (Git 2.23+)
```

```
git switch -c feature/user-auth
```

```
# Create a branch from a specific commit or tag
```

```
git checkout -b hotfix/login-crash v2.1.0
```

Switching Branches

```
# Switch to an existing branch
git checkout main
git switch main          # Git 2.23+ syntax

# Switch back to the previous branch
git checkout -
git switch -

# See which branch you're on
git branch
git status
```

What switching branches does

1. Moves `HEAD` to the new branch pointer
2. Updates the staging area to match the new branch's tree
3. Updates the working directory to match

If you have uncommitted changes that conflict with the target branch, Git will refuse to switch.

Deleting Branches

```
# Delete a branch that has been fully merged  
git branch -d feature/user-auth
```

```
# Force-delete a branch regardless of merge status  
git branch -D feature/abandoned
```

```
# Delete a remote-tracking branch  
git push origin --delete feature/user-auth
```

```
# Rename a branch  
git branch -m old-name new-name
```

Part 3 — Merging

`git merge` — Integrating Branch Work

```
# Switch to the target branch (the one you merge INTO)
git checkout main

# Merge the feature branch into main
git merge feature/user-auth
```

The golden rule of merging

Always merge **into** the target branch. `git merge feature` means: "bring feature's work into my current branch."

Fast-Forward Merge

Happens when the target branch has **no new commits** since the branch was created:

Before:		After (fast-forward):
main: A ← B		main: A ← B ← C ← D
	\	
feature: C ← D		main simply moves forward

```
git checkout main
git merge feature/simple-fix
# Fast-forward merge – no merge commit created
```

A fast-forward is the cleanest merge: linear history, no extra commit.

Preventing Fast-Forward (Preserving Branch History)

```
# Always create a merge commit even if fast-forward is possible  
git merge --no-ff feature/user-auth
```

Without `--no-ff`: A — B — C — D — E (linear)

With `--no-ff`:
A — B ————— M (merge commit M)
 \
 C — D — /

Many teams use `--no-ff` on `main` to preserve the record of which commits came from which feature.

3-Way Merge

Happens when **both branches have diverged** from a common ancestor:

```
common ancestor:  A
main:             A ←—— B ←—— C
feature:          A ←—— D ←—— E

Git finds A (common ancestor) and merges C + E → M
main after merge: A ←—— B ←—— C ←—— M
                  ↑
                D ←—— E ————
```

Git uses the three-way content merge algorithm to produce a result automatically.

Part 4 — Merge Conflicts

When Conflicts Happen

A conflict occurs when:

- **Both branches modified the same lines** of the same file
- Git cannot automatically determine which version to keep

```
$ git merge feature/user-auth
Auto-merging src/auth.js
CONFLICT (content): Merge conflict in src/auth.js
Automatic merge failed; fix conflicts and then commit the result.
```

Reading a Conflict Marker

```
<<<<<<< HEAD
  const timeout = 30 * 60 * 1000; // 30 minutes (current branch)
=====
  const timeout = 60 * 60 * 1000; // 60 minutes (incoming branch)
>>>>>>> feature/user-auth
```

Marker	Meaning
<<<<<<< HEAD	Start of your current branch's version
=====	Divider between the two versions
>>>>>>> feature/user-auth	End of the incoming branch's version

Resolving a Conflict — Step by Step

```
# Step 1 – See which files have conflicts
git status

# Step 2 – Open each conflicted file and choose the resolution
#           (edit out the conflict markers, keep what you want)

# Step 3 – Stage the resolved file
git add src/auth.js

# Step 4 – Complete the merge with a commit
git commit
# (Git pre-fills the message: "Merge branch 'feature/user-auth'")
```

Aborting a Merge

If you change your mind mid-conflict:

```
# Abort the merge and return to pre-merge state  
git merge --abort
```

Conflict Resolution Strategies

```
# Accept all of "ours" (current branch) for a file  
git checkout --ours src/auth.js
```

```
# Accept all of "theirs" (incoming branch) for a file  
git checkout --theirs src/auth.js
```

```
# Use a merge tool (opens configured tool: VS Code, vimdiff, etc.)  
git mergetool
```

VS Code as merge tool

```
git config --global merge.tool vscode  
git config --global mergetool.vscode.cmd 'code --wait $MERGED'
```

Part 5 — Visualizing Branches

git log --graph

```
# Text-based ASCII graph of your branch history
git log --oneline --graph --all
```

```
* a3f7c21 (HEAD -> main) chore: release v2.2.0
*   b2e1d40 Merge branch 'feature/user-auth'
| \
|  * c9a8b12 feat: add JWT refresh endpoint
|  * d7f3e56 feat: implement login/logout
* | e1c2a34 fix: hotfix login redirect
|/
* f0b9c87 chore: setup project structure
```

Useful Log Flags for Branch Visualization

Flag	Effect
<code>--oneline</code>	One line per commit
<code>--graph</code>	ASCII branch graph
<code>--all</code>	Include all branches, not just current
<code>--decorate</code>	Show branch/tag labels (default in most Git versions)
<code>--date=short</code>	Show dates as <code>YYYY-MM-DD</code>
<code>--author="Jane"</code>	Filter by author

```
# Alias for quick branch graph
git config --global alias.lg "log --oneline --graph --all --decorate"
git lg
```

Summary — Branching & Merging

Command	What it does
<code>git branch feature/x</code>	Create a new branch
<code>git checkout -b feature/x</code>	Create and switch to branch
<code>git switch -c feature/x</code>	Create and switch (modern syntax)
<code>git merge feature/x</code>	Merge branch into current branch
<code>git merge --no-ff feature/x</code>	Merge with a merge commit always
<code>git merge --abort</code>	Abort an in-progress merge
<code>git branch -d feature/x</code>	Delete a merged branch
<code>git log --oneline --graph --all</code>	Visualize branch history